

Take-Home Refactoring Team Design Exercise

Software Developer II & Senior Software Developer III

Context

Caseware builds software that supports audit, assurance, risk, and compliance professionals in delivering high-quality work efficiently and consistently across their firm's practice. Our platforms help practitioners plan engagements, analyze complex financial and operational data, apply professional standards and methodologies, and document defensible conclusions.

AI at Caseware is not intended to replace professional judgment. Instead, it augments practitioners by helping them:

- Ask better questions of their data
- Quickly understand context across structured and unstructured sources
- Investigate and explain findings with traceability and confidence
- Apply consistent approaches across engagements and teams.

This exercise evaluates architecture, judgment, and system design at a Senior level. It is used for both Software Developer II and Senior Software Developer III candidates — we calibrate the bar by level, not the problem itself.

The Problem

Our customers work in “Engagement Files” which contain all the work that an accounting firm would do for a customer over the course of a year. These engagement files are created based on “Product Templates,” which are regularly updated by our content teams in different markets.

Engagement files are initially created with the most recent version of a product template, but those templates receive updates throughout the year. Once a product template has an update published, that update needs to be evaluated by our customers. For each of their engagement files, the customer will either apply the update to an engagement or decline the update.

Users need to see at a glance which of their engagement files have pending updates, and users need a summary of the pending inbound changes in order to make that apply/ decline decision. We would like you to design a system that can provide users with that information in an up-to-date manner, for ~100s of engagements per firm across many different products. Product template updates are relatively infrequent, on the order of once per week per product.

Consider how to handle the scenario where multiple template updates accumulate before the user makes the accept/decline choice.

Actually applying the updated template content is outside the scope of this assignment.

Context (Other systems you can work with)

All versions of all product templates are stored in a database, and individual templates can be quickly retrieved given a template ID and a version. Publishing an updated product template involves writing to this database. Creating a new engagement file involves reading from this database in order to retrieve the template file to use to create the new engagement file.

Product templates are essentially zip archives of structured JSON data. You can assume that you have access to a quick and reliable way to compare two different versions of the same Product Template and extract a JSON diff. However, the users are non-technical and would strongly prefer something human-readable.

Notably, the product template database is shared between customer firms, as they all use the same product templates. This database does not currently retain any information about Engagement files, which are stored in their own customer-specific databases.

Engagement files currently store the template ID and version used for their creation. However, the stored form of an engagement is not a directly queryable record of its current state — opening one rehydrates it into memory in the pods serving that session, where volatile financial fields are established. Reading the template information therefore requires loading the engagement, which is a slow operation (~1 minute per file; this should be treated as a hard constraint). The same system which loads engagements is also responsible for handling engagement creation, and for processing the user's accept/decline decision.

You may assume you can add hooks or events to any actions taken by either the product template storage system, or by the engagement management system.

Non-Functional Requirements for Production

Whatever you propose has to hold up under real usage, not just work in a demo with a handful of engagements.

- **Scale:** design for on the order of 100s of engagements per firm, across many different products, and firms of very different sizes on the same system. Template updates are relatively infrequent — on the order of once per week per product — but a single update can affect every firm using that product at once.
- **Performance:** opening a firm's engagement list, and finding out about a newly published update, cannot depend on loading engagement files synchronously — that ~1-minute cost can't show up as a user-facing wait, no matter how many engagements a firm has.
- **Observability:** you should be able to tell, without waiting for a user to report it, how many engagements are affected by a newly published update, whether the pending-update check is actually running for every firm, and what a firm's history of apply/decline decisions has been.
- **Security:** engagement content is client-confidential, and the pending-update data this feature introduces is derived from a template store shared across every firm. Whatever tracks and surfaces that state must enforce firm-level isolation as a structural property (not just an application-layer filter), must restrict who can see or act on a firm's pending updates to users authorized for that firm, and must respect any data-residency requirements the firm's engagement data is already subject to.

Part 1: Design (Primary Focus)

Please provide a short design document (1–3 pages max) that covers:

1. Target Architecture
2. Implementation Plan
3. Testing Strategy
4. Evaluation & Observability
5. Failure Modes & Tradeoffs

Part 2: Targeted Implementation

Optionally (but encouraged), implement one small slice of the system.

Guidelines:

- Focus on interfaces, contracts, and correctness
- Tests or validation logic are a plus

This is not a full application build.

AI Usage (Important)

You may use any AI tools you normally use (e.g., ChatGPT, Copilot, Claude).

During the follow-up review, we will ask you to explain:

- Where AI helped you
- Where you corrected or ignored AI output
- How you would guide other engineers using AI on this system
- Where AI should not be trusted in this domain

We are evaluating judgment and maturity, not just usage.

Assumptions & Constraints

- You may choose any cloud provider or stack (note we are an AWS shop)
- You may state reasonable assumptions
- Optimize for clarity and tradeoffs over completeness

Deliverables

Please submit:

1. Design document (PDF or Markdown)
2. (Optional) Code or repository
3. (Optional) Session History for any AI tooling used
4. Any diagrams you created

What We're Evaluating

- Architecture and system design judgment
- Understanding of LLMs and other AI tools
- Production and operational thinking
- Clarity of communication
- Thoughtful use of AI in the Software Development Process

Time Expectation

We expect this exercise to take no more than 2–3 hours total.

Please do not over-optimize.

Follow-Up

We will review your submission. If we like it, you will be invited to a live design review to discuss decisions, tradeoffs, and your proposed implementation of the design